

Comparison of speed and performance between Python and R implementations of the AddiVortes algorithm

Lewis Maltby

June 30, 2026

1 Introduction

The purpose of this article is to compare the speed and performance of the AddiVortes algorithm between the current up-to-date Python and R implementations. We will benchmark the fitting time of a model, as well as the amount of time required to make simple predictions on a test set, to try to determine which of the two implementations is faster to use. We will also benchmark the in-sample and out-of-sample RMSE on that data to determine which of the two implementations gives predictions with lower errors.

The code used to generate the benchmarking data, as well as the generated data itself and all referenced files, are available to view at <https://github.com/snowy-the-leopard/AddiVortes>. This data was generated using the ADDIVORTES R package, version 0.6.5, available to view here <https://github.com/johnpaulgosling/AddiVortes>, and the ADDIVORTES Python package, version 0.6.5, available to view here <https://github.com/johnpaulgosling/py-addivortes>.

2 Benchmarking methodology

To ensure fairness, consistency, and accuracy with these tests, the following controls were implemented on these benchmarks:

1. The benchmarks were run on a new GitHub Codespace, a clean installation of Python and R were used, with only the necessary packages downloaded to minimise background resource usage. The Codespace used is a 2-core 2.8GHz system with 8GB of memory. Exact details of hardware used can be found at [AddiVortes\mySpecs.html](#).
2. The code was set up such that each implementation used only a single thread, to counteract the potentially different methods of multi threading between the two languages. This choice was made as we are working under the assumption that when we are working with HPC resources, the code will be optimised to allow multiple threads to be used at once. This choice allows us to compare algorithmic efficiency in each implementation more directly.
3. The same models and predictions were fitted repeatedly in order to capture the small variations in fit and prediction times between runs. This will also produce a set of slightly varying prediction results due to the random nature of the MCMC fitting algorithm, and allow us to better analyse how consistent our results are.
4. The benchmarks were run sequentially rather than simultaneously to minimise the risk of shared resources skewing the results, where one script could take computational priority over another, leading to a larger discrepancy in results.
5. The timings were taken only between the computation required for fitting and generating predictions, with the pre-generated datasets already loaded in prior to this.

3 Datasets used in benchmarking

To allow for a wide scope of comparison between the two implementations, the following 4 different datasets were used in benchmarking:

3.1 Boston dataset

The Boston Housing Dataset, derived from information from the U.S. Census Service. This consists of 13 numerical covariates, and we model it using the following fixed parameters, with the rest of the parameters taking their default values: `m = 50`

3.2 Synthetic dataset

We have constructed a synthetic dataset using noisy functions of independently and identically distributed $U[0, 1]$ observations. This consists of 5 numerical covariates, and we model it by taking all the parameters as their default values.

3.3 Spherical dataset

We have constructed random longitudes and latitudes to represent positions on a sphere, and a noisy response function. This consists of 2 numerical covariates (the coordinates), and we model it using the following fixed parameters, with the rest of the parameters taking their default values: `m = 50`, `totalMCMCiter = 500`, `mcmcBurnIn = 100`, `metric = "S"`. Note that this uses great circle distance, the shortest distance between two points over the surface of the sphere.

3.4 Categorical dataset

We have constructed this dataset using random numerical and categorical covariates, as well as a noisy response function. This consists of 2 numerical and 2 categorical covariates, and we model it using the following fixed parameters, with the rest of the parameters taking their default values: `m = 50`, `totalMCMCiter = 500`, `mcmcBurnIn = 100`, `catScaling = 1`

4 Benchmark results

The full benchmarks, which carry out model fitting and prediction on the above 4 datasets, were run for 106 iterations each. The code can be found at `AddiVortes\tests.R` and `AddiVortes\tests.py`

Table 1: Mean Fit and Prediction Times (R vs Python)

Test	Fit Mean (R)	Fit Mean (Python)	Fit % Diff	Pred Mean (R)	Pred Mean (Python)	Pred % Diff
Test 1	14.800	12.616	-14.760	10.236	4.319	-57.800
Test 2	1.813	1.412	-22.145	1.910	0.731	-61.754
Test 3	3.376	3.353	-0.682	2.671	2.108	-21.100
Test 4	0.655	0.460	-29.781	0.662	0.247	-62.721

Table 2: Standard Deviation of Fit and Prediction Times (R vs Python)

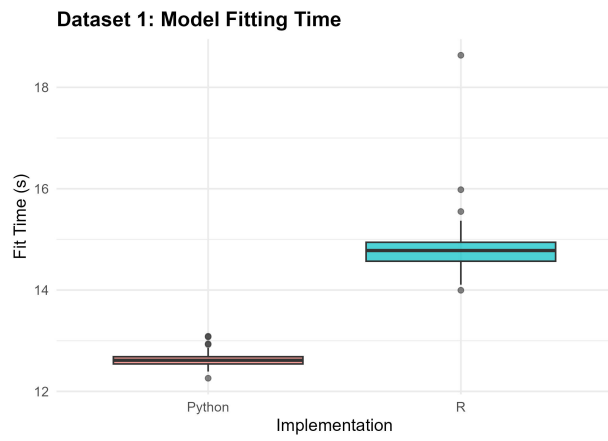
Test	Fit SD (R)	Fit SD (Python)	Fit % Diff	Pred SD (R)	Pred SD (Python)	Pred % Diff
Test 1	0.486	0.133	-72.709	1.146	0.071	-93.844
Test 2	0.347	0.059	-83.076	0.090	0.043	-52.488
Test 3	0.265	0.089	-66.307	0.605	0.065	-89.185
Test 4	0.145	0.041	-71.812	0.142	0.042	-70.497

Table 3: Mean In-sample and Out-of-sample RMSE (R vs Python)

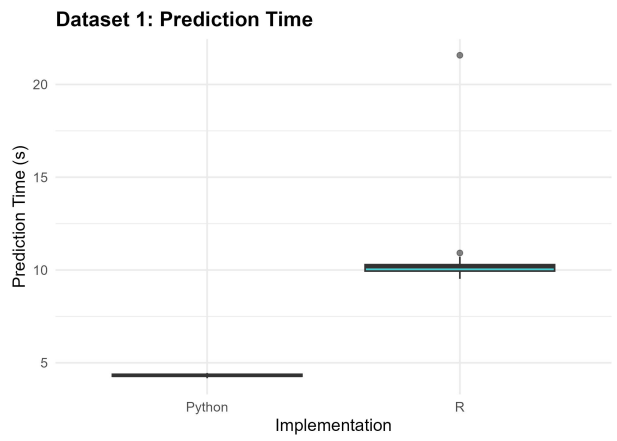
Test	iRMSE Mean (R)	iRMSE Mean (Python)	iRMSE % Diff	oRMSE Mean (R)	oRMSE Mean (Python)	oRMSE % Diff
Test 1	0.777	0.792	1.983	3.674	3.674	0.001
Test 2	1.029	1.081	4.996	1.374	1.419	3.301
Test 3	1.215	1.236	1.722	4.537	2.223	-51.013
Test 4	2.240	2.247	0.327	3.027	3.043	0.507

Table 4: Standard Deviation of RMSE (R vs Python)

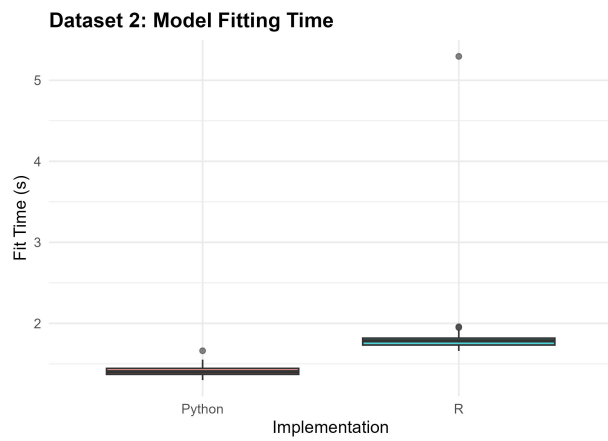
Test	iRMSE SD (R)	iRMSE SD (Python)	iRMSE % Diff	oRMSE SD (R)	oRMSE SD (Python)	oRMSE % Diff
Test 1	0.033	0.038	14.582	0.102	0.104	2.328
Test 2	0.082	0.081	-1.423	0.194	0.120	-38.232
Test 3	0.048	0.049	1.242	0.567	0.048	-91.523
Test 4	0.038	0.036	-4.641	0.067	0.067	-0.978



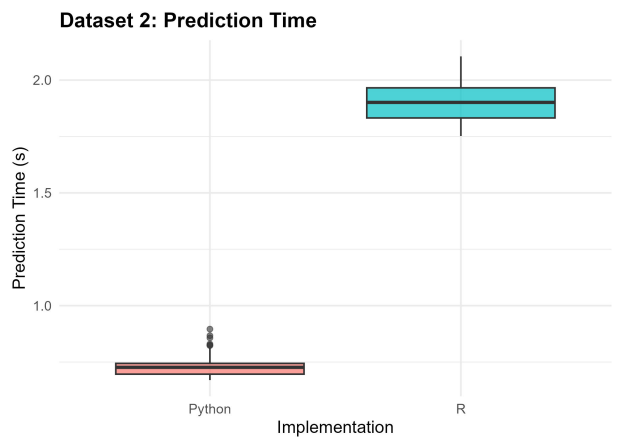
(a) Fit 1



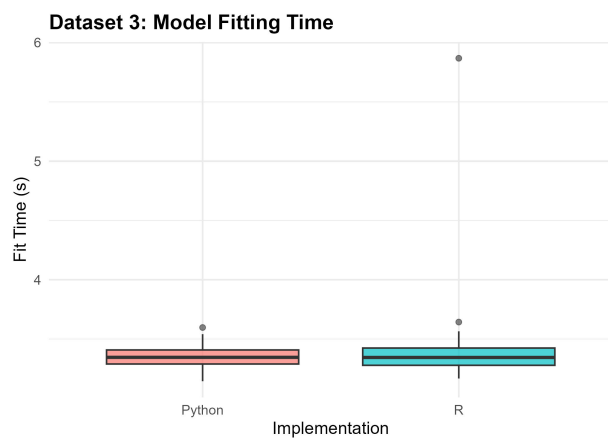
(b) Pred 1



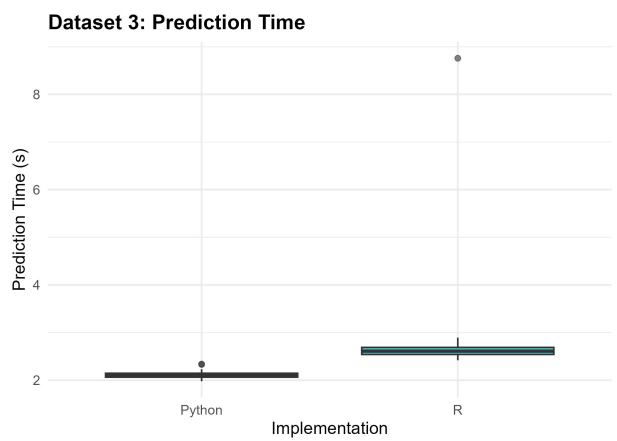
(c) Fit 2



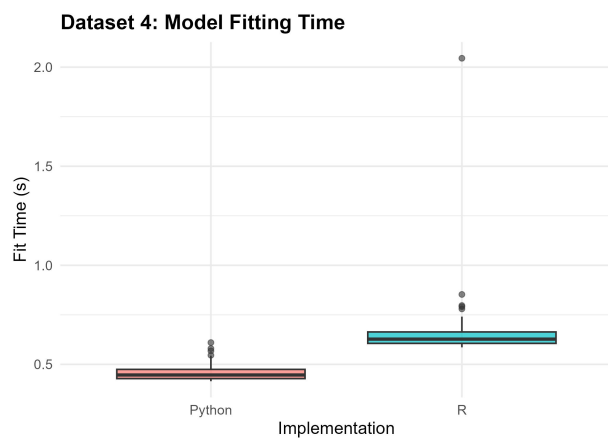
(d) Pred 2



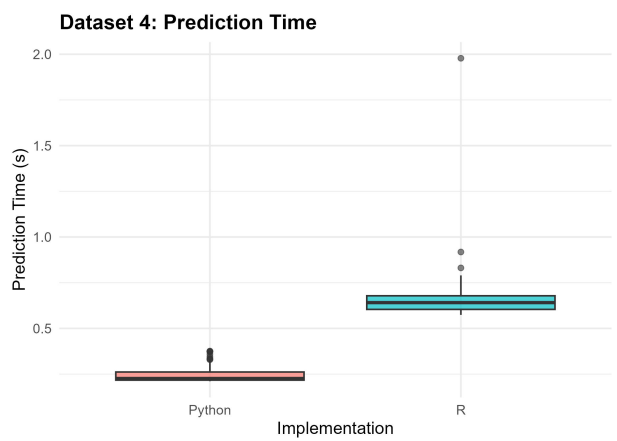
(e) Fit 3



(f) Pred 3



(g) Fit 4

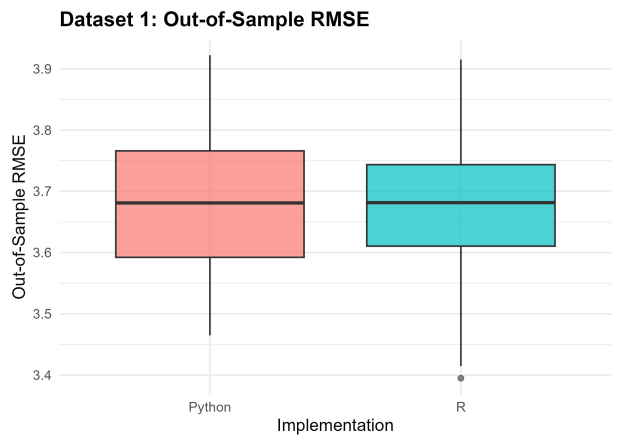


(h) Pred 4

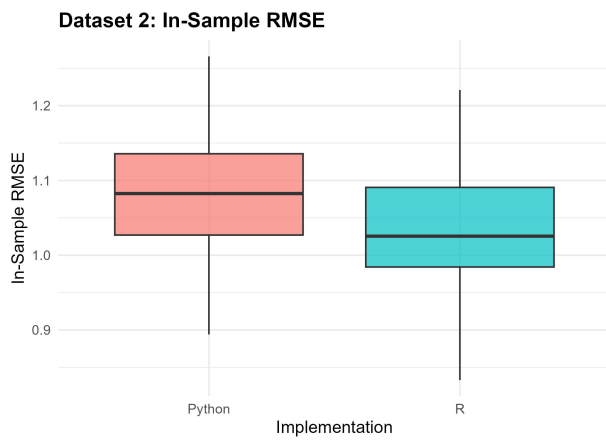
Figure 1: Model fitting and prediction times across datasets.



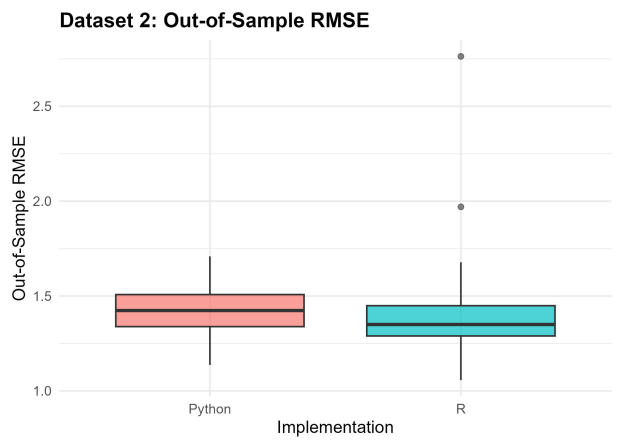
(a) iRMSE 1



(b) oRMSE 1



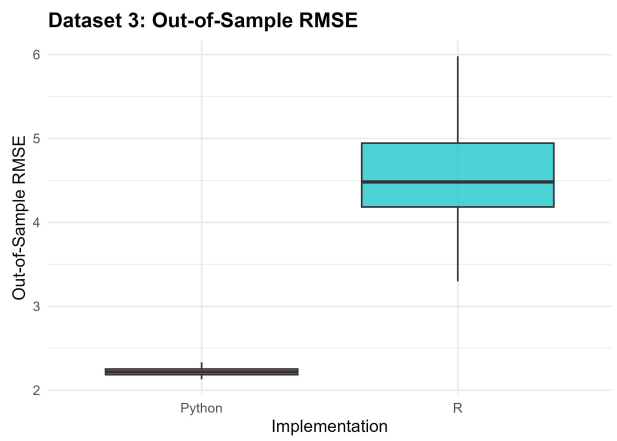
(c) iRMSE 2



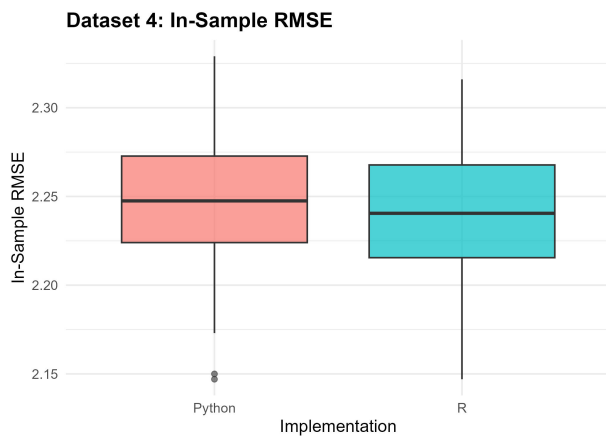
(d) oRMSE 2



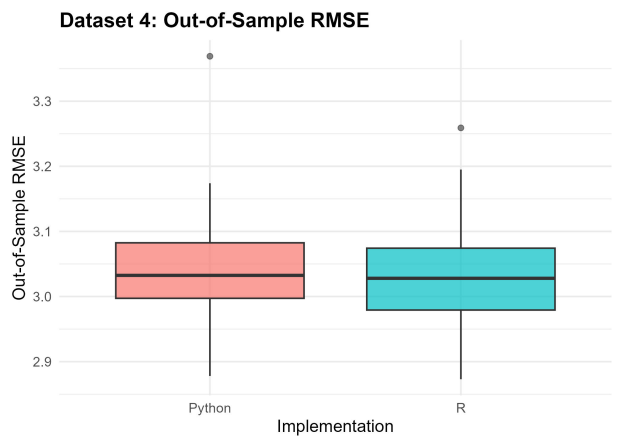
(e) iRMSE 3



(f) oRMSE 3



(g) iRMSE 4



(h) oRMSE 4

Figure 2: In-sample and out-of-sample RMSE across datasets.

5 Analysis of results

5.1 Fit times

The mean fit time using Python was lower for all models, and substantially lower for all models, excluding model 3 (spherical). With an average decrease of 22% in mean fit time using Python, we can certainly conclude that the Python implementation is significantly faster than R at fitting the AddiVortes model using a Euclidean metric. The standard deviation of fit times using Python was significantly lower for all models, with an average decrease of 73% when using Python instead of R.

Therefore in terms of both lower and more consistent fit times, Python is superior.

5.2 Prediction times

The mean prediction time using Python was significantly lower for all models, with an average decrease of 51% when using Python instead of R. The standard deviation of fit times using Python was significantly lower for all models, with an average decrease of 77% when using Python instead of R.

Therefore in terms of both lower and more consistent fit times, Python is superior.

5.3 In-sample RMSEs

The mean in-sample RMSE using Python was slightly higher for all models, with an average increase of 2% when using Python instead of R. The standard deviation of in-sample RMSEs using Python gave mixed results in comparison to R. The standard deviation of in-sample RMSEs on model 1 was significantly higher. This model, representing real world data that could be more unstructured than the synthetic datasets we generated, should be the most significant for us here, where the other differences vary in sign.

Therefore in terms of lower and more consistent in-sample RMSEs, we should take a slight preference for R, with its slightly, but consistently, better mean in-sample RMSEs, and its proficiency for more consistent in-sample RMSEs with the real-world dataset used.

5.4 Out-of-sample RMSEs

The mean out-of-sample RMSE using Python gave mixed results in comparison to R. The mean out-of-sample RMSE was almost the same for models 1 and 4, with only a slight increase for model 2. This shows that R also has a slightly lower mean out-of-sample RMSE using a Euclidean metric. On the other hand, the out-of-sample RMSE was significantly lower for model 3 (spherical), with a reduction of 51% in mean value. The standard deviation of out-of-sample RMSEs using Python was significantly lower for models 2 and 3, with models 1 and 4 having much smaller changes with varying sign. This demonstrates that Python can generate much more consistent out-of-sample RMSEs.

Therefore in terms of out-of-sample RMSEs, it seems that R produces a consistently slightly lower mean, when using the Euclidean metric, however Python seems to be able to produce much more consistent values, and produces a much lower mean out-of-sample RMSE when using the Spherical metric. Hence it is hard to demonstrate any clear preference we should have of either implementation in this regard.

6 Conclusion

Given that the results obtained using Python are significantly faster and more consistent, and there is only evidence of a very slight reduction in RMSEs when using R, I would recommend the Python implementation for further use and testing.